UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ASIGNATURA:

Metodologías de Desarrollo de Software

TEMA:

Paradigmas del desarrollo de software

DOCENTE:

Ing. Kevin Jair Chuquitarco Velasco

ESTUDIANTE:

Jeffry David Vera Borja

NIVEL – NRC:

Tercero - 29531

PERIODO:

S-I ABR 26 - AGO 26

FECHA DE ENTREGA:

19/04/2026

Tabla de Contenido

Paradigmas del desarrollo de software 3
Introducción 3
Objetivos 3
Desarrollo 4
Modelo en Cascada 4
Modelo en V 5
Modelo Incremental 7
Modelo Iterativo 8
Modelo Espiral 8
Conclusiones 10
Referencias 11

Paradigmas del desarrollo de software

Introducción

Los paradigmas del desarrollo de software representan enfoques estructurados para la creación de aplicaciones, evolucionando desde métodos lineales tradicionales hacia procesos flexibles y adaptativos. Su importancia radica en la necesidad de gestionar complejidad, riesgos y cambios en requisitos en un contexto donde el software es esencial para industrias modernas. El propósito de este trabajo es analizar los principales modelos, compararlos y evaluar sus ventajas y desventajas para orientar la selección adecuada en proyectos universitarios o profesionales.

Objetivos

Objetivo General

Examinar los paradigmas tradicionales del desarrollo de software, sus características fundamentales y aplicaciones prácticas para comprender su impacto en la estructuración y eficiencia de proyectos informáticos..

Objetivos Específicos

- Describir detalladamente los modelos de ciclo de vida clásicos: cascada, en V, incremental, iterativo y espiral, destacando sus fases y principios operativos.
- Comparar las ventajas y desventajas de cada modelo tradicional en términos de gestión de riesgos, flexibilidad ante cambios y entrega de valor al cliente.
- Evaluar los criterios de selección de estos paradigmas según la complejidad del sistema y la estabilidad de los requisitos iniciales.

Desarrollo

Los paradigmas del desarrollo de software son marcos conceptuales y metodológicos que orientan la forma en que los equipos planifican, construyen, prueban y entregan sistemas informáticos. Cada modelo propone una filosofía distinta respecto a la organización del trabajo, la gestión del riesgo y la relación con el cliente. A continuación, se describen en detalle los cinco principales modelos de desarrollo de software (Pressman y Maxim, 2015).

Modelo en Cascada

El modelo en cascada, propuesto formalmente por Winston W. Royce en 1970, fue uno de los primeros intentos de sistematizar el proceso de desarrollo de software. Su nombre proviene de la representación visual del flujo de trabajo: cada fase desciende hacia la siguiente como el agua en una cascada, sin posibilidad de retorno una vez completada una etapa (Sommerville, 2016). Este modelo sigue una secuencia lineal y estricta de fases claramente diferenciadas.

Las fases del modelo son: (1) análisis y definición de requisitos, donde se documentan todas las necesidades del sistema; (2) diseño del sistema, que define la arquitectura, componentes e interfaces; (3) implementación y pruebas unitarias, donde se codifican los módulos individuales; (4) integración y pruebas del sistema, que verifica el funcionamiento conjunto; y (5) operación y mantenimiento, fase que inicia una vez entregado el producto. Cada etapa debe completarse y aprobarse mediante documentación formal antes de iniciar la siguiente (Pressman y Maxim, 2015).

Tabla 1

Ventajas y desventajas del modelo en cascada

Aspecto	Ventajas	Desventajas
Estructura y orden	Documentación clara y completa	Sin flexibilidad ante cambios
Gestión del cambio	Fácil seguimiento de avances	Cambios tardíos son costosos
Participación del cliente	Requisitos bien definidos desde el inicio	El cliente no ve resultados hasta el final
Tipo de proyecto ideal	Proyectos con requisitos estables	No apto para proyectos dinámicos

Nota. Comparación de características del modelo en cascada. Adaptado de *Ingeniería del software: Un enfoque práctico*, por Pressman & Maxim (2015).

Modelo en V

El modelo en V, también denominado modelo de verificación y validación, es una extensión del modelo en cascada que enfatiza la correspondencia entre cada fase de desarrollo y su actividad de prueba asociada. Su representación gráfica forma una letra "V": el lado izquierdo corresponde a las fases de desarrollo descendentes, y el lado derecho a las fases de prueba ascendentes (Bourque y Fairley, 2014).

En el lado izquierdo se definen: los requisitos del sistema, los requisitos del software, el diseño arquitectónico y el diseño detallado. En la parte inferior, la codificación une ambos lados. En el lado derecho, espejando cada etapa izquierda, se ejecutan: pruebas unitarias, pruebas de

integración, pruebas del sistema y pruebas de aceptación. Esta correspondencia garantiza que cada componente del sistema sea validado desde su concepción, reduciendo considerablemente la aparición tardía de defectos (Sommerville, 2016).

Este modelo es ampliamente utilizado en sistemas donde la fiabilidad es crítica, como software médico, aeroespacial o financiero. Su principal limitación, al igual que el modelo en cascada, es la dificultad para adaptarse a cambios una vez iniciado el proceso de desarrollo (Pressman y Maxim, 2015).

Tabla 2

Correspondencia entre fases de desarrollo y prueba en el modelo en V

Fase de Desarrollo (lado izquierdo)	Fase de Prueba correspondiente (lado derecho)
Requisitos del sistema	Pruebas de aceptación
Requisitos del software	Pruebas del sistema
Diseño arquitectónico	Pruebas de integración
Diseño detallado	Pruebas unitarias
Codificación (vértice inferior de la V)	

Nota. Adaptado de Sommerville (2016).

Modelo Incremental

El modelo incremental divide el desarrollo del sistema en una serie de incrementos, cada uno de los cuales añade funcionalidad al anterior. A diferencia del modelo en cascada, el cliente recibe versiones parciales pero operativas del software desde etapas tempranas del proyecto. Cada incremento pasa por las fases de análisis, diseño, codificación y prueba, pero aplicadas únicamente al módulo o funcionalidad que se está desarrollando en esa entrega (Pressman y Maxim, 2015).

El proceso comienza con la definición de todos los requisitos del sistema, pero su implementación se distribuye en múltiples entregas planificadas. El primer incremento generalmente incluye las funciones esenciales del producto (núcleo del sistema); los siguientes añaden características adicionales hasta completar el sistema total. Este enfoque reduce el riesgo de fracaso del proyecto al detectar problemas en fases tempranas y permite priorizar las funcionalidades de mayor valor para el cliente (Sommerville, 2016).

Tabla 3

Estructura de entregas en el modelo incremental

Incremento	Funcionalidades incluidas	Estado del sistema
Incremento 1	Núcleo del sistema (funciones básicas)	Versión funcional inicial
Incremento 2	Módulos secundarios de mayor prioridad	Sistema ampliado y mejorado

Incremento N	Funcionalidades restantes del sistema	Sistema completo y entregado

Nota. Adaptado de Pressman & Maxim (2015).

Modelo Iterativo

El modelo iterativo se basa en la repetición de ciclos de desarrollo denominados iteraciones. Cada iteración produce una versión mejorada del sistema, incorporando retroalimentación del cliente y correcciones de los ciclos anteriores. A diferencia del modelo incremental, que añade nuevas funcionalidades en cada ciclo, el modelo iterativo puede revisitar y refinar funcionalidades ya desarrolladas (Larman, 2004).

Cada iteración comprende cuatro fases: (1) planificación, donde se establecen los objetivos del ciclo; (2) análisis y diseño, que revisa y actualiza la arquitectura del sistema; (3) implementación, donde se codifica y prueba lo planificado; y (4) evaluación, en la que el cliente revisa los resultados y proporciona retroalimentación para la siguiente iteración. El proceso continúa hasta que el sistema cumple los requisitos establecidos (Sommerville, 2016).

Este modelo es especialmente adecuado cuando los requisitos del sistema no están completamente definidos al inicio del proyecto, o cuando se espera que evolucionen a lo largo del tiempo. Su principal ventaja radica en la detección temprana de errores y la capacidad de adaptación continua; sin embargo, puede ser difícil controlar su alcance si no se gestiona adecuadamente (Pressman y Maxim, 2015).

Modelo Espiral

El modelo espiral fue propuesto por Barry Boehm en 1988 y combina la naturaleza iterativa del desarrollo con los mecanismos de control de riesgo sistemático del modelo en

cascada. Su estructura se representa como una espiral que crece hacia afuera, donde cada vuelta completa representa un ciclo de desarrollo y conlleva un análisis explícito de riesgos antes de avanzar a la siguiente fase (Boehm, 1988).

Cada ciclo de la espiral se divide en cuatro cuadrantes: (1) determinación de objetivos, en que se definen las metas del ciclo, las restricciones y alternativas; (2) evaluación y reducción de riesgos, donde se identifican, analizan y mitigan los principales riesgos del proyecto, frecuentemente mediante la construcción de prototipos; (3) desarrollo y validación, en que se selecciona y aplica el modelo de desarrollo más adecuado (cascada, incremental, etc.) para el ciclo actual; y (4) planificación, donde se revisa el progreso y se planifica el siguiente ciclo (Pressman y Maxim, 2015).

El modelo espiral es idóneo para proyectos grandes, complejos y de alto riesgo, como sistemas de defensa, aeroespaciales o financieros de gran escala. Su principal fortaleza es la gestión proactiva del riesgo; sin embargo, requiere experiencia considerable para identificar y gestionar los riesgos correctamente, y puede resultar costoso en tiempo y recursos si se aplica a proyectos pequeños (Sommerville, 2016).

Tabla 5

Cuadrantes del modelo espiral

Cuadrante	Actividad principal	Propósito
Objetivos	Definición de metas y restricciones	Establecer qué se quiere lograr en el ciclo
Riesgos	Identificación y mitigación de riesgos	Reducir la incertidumbre antes de avanzar

Desarrollo	Construcción y validación del producto	Producir un incremento funcional del sistema
Planificación	Revisión y preparación del siguiente ciclo	Decidir si continuar o concluir el proyecto

Nota. Adaptado de Boehm (1988) y Pressman & Maxim (2015).

Conclusiones

El análisis de los cinco modelos de desarrollo de software permite concluir que no existe un paradigma universalmente superior: cada modelo responde a un conjunto específico de condiciones de proyecto. El modelo en cascada es adecuado cuando los requisitos son estáticos y bien comprendidos desde el inicio, mientras que el modelo en V añade una dimensión de calidad al vincular cada fase de desarrollo con su correspondiente actividad de prueba, siendo ideal para sistemas de alta criticidad.

El modelo incremental y el iterativo ofrecen mayor flexibilidad al permitir entregas parciales y la incorporación de retroalimentación continua, lo que los convierte en opciones sólidas para proyectos donde los requisitos evolucionan. El modelo espiral, por su parte, representa la opción más robusta para proyectos de gran envergadura con alto nivel de incertidumbre, gracias a su enfoque sistemático de gestión de riesgos en cada ciclo.

En conclusión, el profesional en tecnologías de la información debe conocer en profundidad las características, ventajas y limitaciones de cada paradigma para seleccionar el modelo que mejor se adapte a las necesidades específicas del proyecto, el equipo de trabajo y las

expectativas del cliente, ya que una elección inadecuada puede comprometer la calidad, el costo y el plazo de entrega del sistema desarrollado.

Referencias

Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, *21*(5), 61–72. https://doi.org/10.1109/2.59

Bourque, P., & Fairley, R. E. (Eds.). (2014). *Guide to the software engineering body of knowledge (SWEBOK v3.0)*. IEEE Computer Society Press.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Pressman, R. S., & Maxim, B. R. (2015). *Ingeniería del software: Un enfoque práctico* (8.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Ingeniería de software* (10.ª ed.). Pearson Educación.